AVP32F335 以 60M 主频在线烧录 Flash 的方法

作者：唐　佳

前　言：

AVP32F335 采用了不同于 TMS320F28335 的不同工艺 FlashIP，由于时序不一样，且在不同延时情况下，Flash 电荷累积数量会直接影响苛刻环境温度下 Flash 的可靠稳定性。经高低温等多种测试得出，在烧录时，CPU 主频在 40M~60M 可保障 Flash 数据的稳定可靠，例程 F28335_FlashAPI 给出了主频在 60M 情况下的 FlashAPI 操作指南，如下列表所示。

Flash API 检查列表：

1. 包含头文件 Flash2833x_API_Config.h，并且在头文件中，修改指定设备的宏定义：

```
#define FLASH_F28335   1
```

修改 CPU 的时钟频率：

```
#define CPU_RATE    16.667L // 修改为 60MHz CPU 主频
```

注意：CPU_RATE 在 DSP2833x_Examples.h 头文件中同样有定义，均需修改为同一值，或保留一个，建议保留头文件 DSP2833x_Examples.h 中的定义。

2. 包含头文件 Flash2833x_API_Library.h。
3. 添加 AVP32F335 的 Flash API 库文件 AVP32F335_API_V210.lib 到工程中。
4. 修改头文件 DSP2833x_Examples.h 中 DSP28_PLLCR 和 DSP28_DIVSEL 宏定义值，使主频工作在 60MHz（40MHz~60MHz 均为建议值，根据外部晶体频率，可能设置的频率不一样）。推荐如下表格所示配置（也可配置成其他主频）：

```
#define DSP28_DIVSEL     2 // Enable /2 for SYSCLKOUT
#define DSP28_PLLCR      4
```

外部晶振频率	DSP28_PLLCR	DSP28_DIVSEL	CPU_RATE (ns)	主频 (MHz)
20M	5	2	20	50MHz
24M	4	2	20.833	48MHz
30M	4	2	16.667	60MHz

5. 确保 PLL 不是运行在坡行模式（即时钟丢失状态位为 0）。如下面程序所示，TI 建议在时钟丢失时，直接关机重启（SystemShutdown(); 函数执行建议）。在 AVP32F335 的 PLL 模块中，由于环境温度的敏感因素，可能导致在外部晶振起振正常的情况下，发生时钟丢失，引起 PLL 进入坡行模式。所以在程序判断到时钟丢失状态时，需要关闭时钟

丢失检测电路。程序如下：

```
//
// Make sure the PLL is not running in limp mode
//
if (SysCtrlRegs.PLLSTS.bit.MCLKSTS != 0)
{
    //
    // Missing external clock has been detected
    // Replace this line with a call to an appropriate
    // SystemShutdown(); function.
    //

    /**
     * 时钟丢失状态位产生会导致内核复位。引起无法烧录；无法配置PLLCR等问题
     * 所以，当状态位产生时，关闭时钟丢失检测电路并清楚时钟丢失状态。
     */
    EALLOW;
    SysCtrlRegs.PLLSTS.bit.MCLKOFF = 1; // 关时钟丢失检测
    SysCtrlRegs.PLLSTS.bit.MCLKCLR = 1; // 清时钟丢失状态
    EDIS;

    asm("        ESTOP0");
}
```

```
    /**
     * 说明：由于PLL电路问题，时钟丢失检测关闭后不再打开
     */
//  EALLOW;
//  SysCtrlRegs.PLLSTS.bit.MCLKOFF = 0;
//  EDIS;
```

注意：随后代码中的时钟丢失检测打开代码，需要注释掉。

6. 【关键】Flash API 函数必须在内部 SARAM 中执行，因此在 flash 中的程序，需要搬移到 RAM 中，如下：

```
// 重要：Flash API必须在0等待的RAM中执行
MemCopy(&Flash28_API_LoadStart,&Flash28_API_LoadEnd,&Flash28_API_RunStart);
```

7. 初始化 CPU 和 Flash 之间的比例因子，32 位全局变量 Flash_CPUScaleFactor。

```
    /**
     * Flash_CPUScaleFactor 在Lib文件中定义，在Flash2833x_API_Library.h中
声明
     * 初始化用到 SCALE_FACTOR 在Flash2833x_API_Config.h中被定义成
     * #define SCALE_FACTOR  1048576.0L*( (200L/CPU_RATE) )  // IQ20
     * 且不可修改，注意修改CPU_RATE匹配。
     */
```

```
    Flash_CPUScaleFactor = SCALE_FACTOR;
```

8. 初始化全局回调函数指针 Flash_CallbackPtr，也可以赋值 NULL（需包含<stdio.h>头文件）。

```
    /**
     * Flash_CallbackPtr 函数指针在Flash2833x_API_Library.h文件中声明，需在main中初始化
     * 成自定义的回调函数，例如：MyCallbackFunction()。并且函数MyCallbackFunction()只能
     * 在RAM中执行，否则会引起异常，程序跑飞。
     * 也可以给回调函数指针赋值NULL，(Flash_CallbackPtr = NULL;)。
     * 注意：NULL在标准头文件 <stdio.h> 中定义，需包含头文件 <stdio.h>
     */
    Flash_CallbackPtr = &MyCallbackFunction;
```

```
/**
 * 回调函数，赋值给 Flash_CallbackPtr
 * 注意：需在RAM中执行
 */

#ifdef __TI_COMPILER_VERSION__
    #if __TI_COMPILER_VERSION__ >= 15009000
        #pragma CODE_SECTION(MyCallbackFunction, ".TI.ramfunc");
    #else
        #pragma CODE_SECTION(MyCallbackFunction, "ramfuncs");
    #endif
#endif

void MyCallbackFunction(void)
{
    // 在Flash API执行过程中处理的时间关键事件，例如：看门狗喂狗
}
```

注意：回调函数本身需在 RAM 中执行，是在 FlashAPI 执行过程中（未完全执行完毕），由 FlashAPI 函数自行调用，完成时间关键性的操作。

9. 【关键】调用 Flash API 函数之前，关闭全局中断。例程中 **InitSysCtrl();** 函数先 **DisableDog();**后续代码 **InitPieCtrl();**执行了 **DINT;**
10. 【可选】采用 **Flash_ToggleTest();**函数判断频率配置是否正确，在 GPIO 脚观察到的频率应该是 10kHz 表示 FlashAPI 主频配置正确。
11. 【有密码时必须执行】解锁 CSM；
12. 调用 API 函数。API 在执行时，
    a) 会关闭看门狗
    b) 检查 0x0882 的 CLASSID 和 0x380090 的 PARTID 是否和匹配。（当出现 PartID

错误时，建议在 MemoryBrowse 窗口查看这两个值）

c) 检查 0x3FFFB9 位置的硅晶版本和 API 版本是否兼容。

d) 关闭和恢复全局中断（INTM,DBGM,XNMICR）。

e) 如果 Flash_CallbackPtr != NULL，则调用回调函数。

f) 返回在 F2833x_API_Config.h 中定义的错误代码。